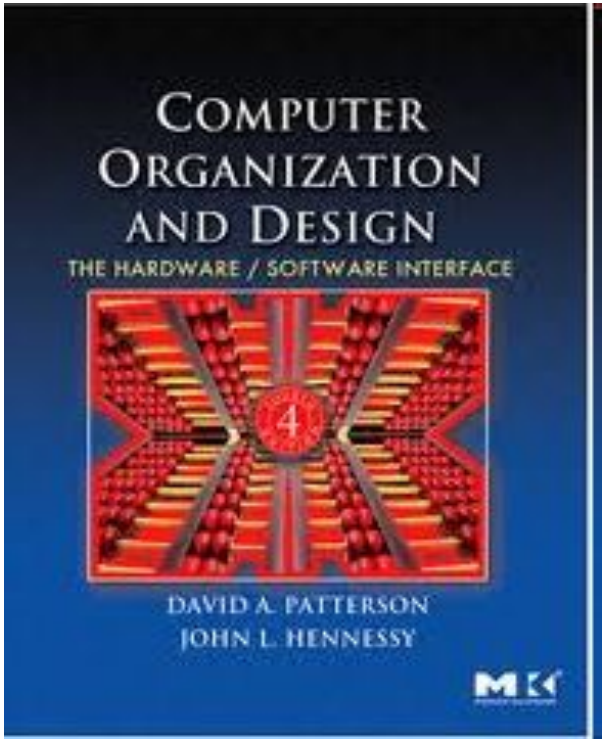
Topics

- Instructions & MIPS instruction set
- Where are the operands ?
- Machine language
- Assembler
- Translating C statements into Assembler
- For details see the book (ch 2):



Main Types of Instructions

- Arithmetic
  - Integer
  - Floating Point
- Memory access instructions
  - Load & Store DMR -> load si store
- Control flow
  - Jump
  - Conditional Branch
  - Call & Return

MIPS arithmetic

- Most instructions have 3 operands
- Operand order is fixed (destination first)

Example:

C code: `A = B + C`

mnemonic <rd><rs1><rs2>

MIPS code: `add $s0, $s1, $s2` s0 = s1 + s2

($s0, $s1 and $s2 are associated with variables by compiler)

MIPS arithmetic

C code: `A = B + C + D;`
`E = F - A;`

MIPS code:

```
add $t0, $s1, $s2
add $s0, $t0, $s3
sub $s4, $s5, $s0
```

a b c d

t0 = s1 + s2

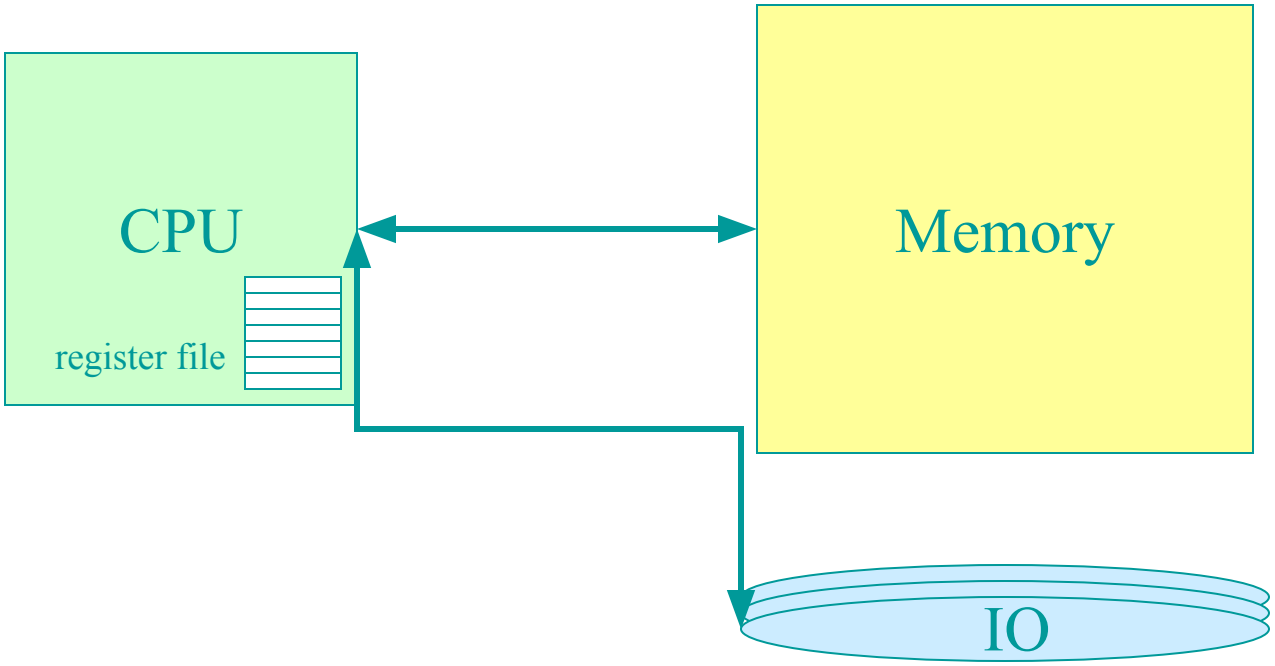
s0 = t0 + s3

s4 = s5 - s0

- Operands must be registers, only 32 registers provided
- Design Principle: *smaller is faster*. Why?

Registers vs. Memory

- Arithmetic instruction operands must be registers, — only 32 registers provided
- Compiler associates variables with registers
- What about programs with lots of variables ?



Register allocation

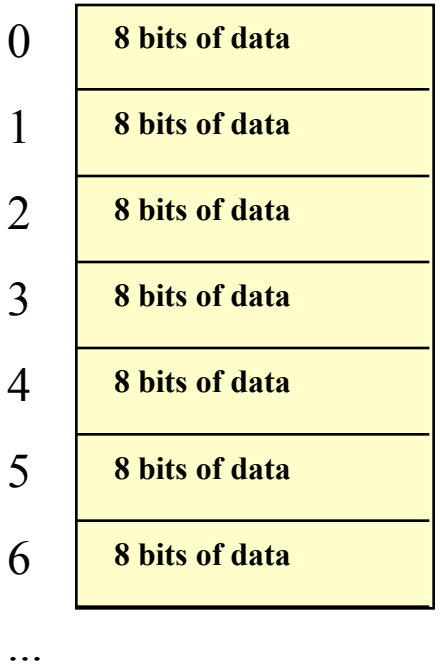
- Compiler tries to keep as many variables in registers as possible

- Some variables can not be allocated
  - large arrays (too few registers)
  - aliased variables (variables accessible through pointers in C)
  - dynamic allocated variables
    - heap
    - stack

- Compiler may run out of registers => ***spilling***

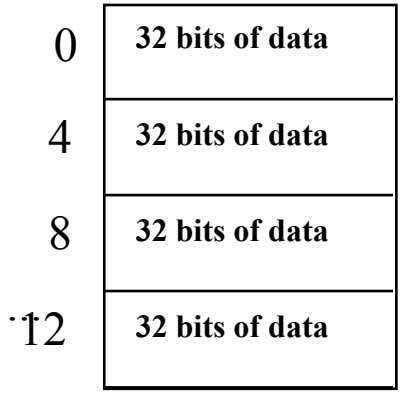
Memory Organization

- Viewed as a large, single-dimension array, with an address
- A memory address is an index into the array
- "Byte addressing" means that successive addresses are one byte apart



Memory Organization

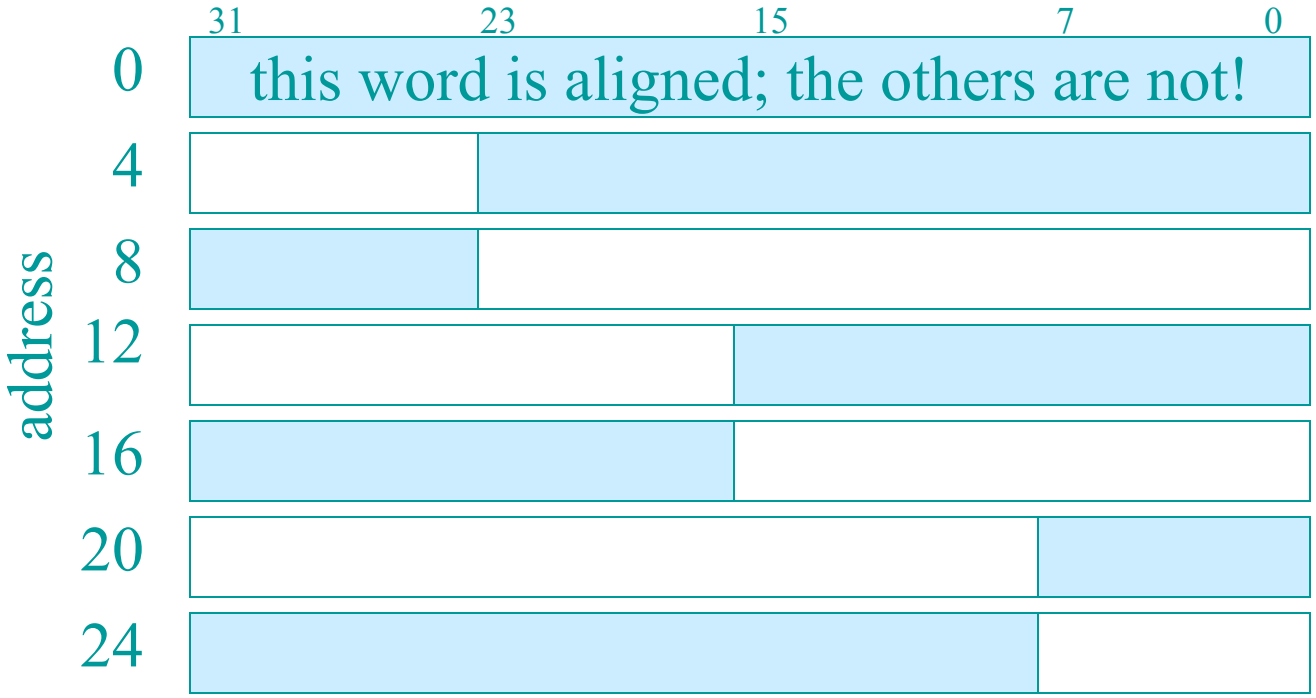
- Bytes are nice, but most data items use larger "words"
- For MIPS, a word is 32 bits or 4 bytes.



Registers hold 32 bits of data

- 2^{32} bytes with byte addresses from 0 to 2^{32}-1
- 2^{30} words with byte addresses 0, 4, 8, ... 2^{32}-4

Memory layout: Alignment



Words are aligned

- What are the least 2 significant bits of a word address?

Instructions: load and store

Example:

C code: `A[8] = h + A[8];`

MIPS code:

```
lw   $t0, 32($s3)
add  $t0, $s2, $t0
sw   $t0, 32($s3)
```

load t0 <- s3(adresa primului el din vector)
32(s3) -> A[0] + 4 * index

t0 = s2(h) + t0

store(merge inapoi in memorie)
t0 -> A[8]; practic faci A[8] = t0

- Store word operation has no destination (reg) operand
- Remember arithmetic operands are registers, not memory!

Used MIPS compiler conventions

Name	Register number	Usage
`$zero`	0	the constant value 0
`$v0-$v1`	2-3	values for results and expression evaluation
`$a0-$a3`	4-7	arguments
`$t0-$t7`	8-15	temporaries
`$s0-$s7`	16-23	saved (by callee)
`$t8-$t9`	24-25	more temporaries
`$gp`	28	global pointer
`$sp`	29	stack pointer
`$fp`	30	frame pointer
`$ra`	31	return address

Our First C code translated

- Can we figure out the code?

```
swap(int v[], int k);
{ int temp;
   temp = v[k]
   v[k] = v[k+1];
   v[k+1] = temp;
}
```

```
128:jal swap // j 256 , $ra = 132
132:...
.....
256: swap:
   muli $2 , $5, 4
   add  $2 , $4, $2
   lw   $15, 0($2)
   lw   $16, 4($2)
   sw   $16, 0($2)
   sw   $15, 4($2)
   jr   $31
```

reg 5 * 4 -> aflam offset

asa aflam adresa lui v[k]

load in 15 pe v[k]

load in 16 pe v[k + 1] pt ca e + 4 fata de v [k]

store la fel ca sus

jump return to 132

Explanation:

index k : $5

base address of v: $4

address of v[k] is $4 + 4.$5

So far we've learned:

- MIPS
  — loading words but addressing bytes
  — arithmetic on registers only

- Instruction Meaning
  //PC = 0 program counter pc -> tine cont de adresa instructiunii curente care se executa
  0: `add $s1, $s2, $s3` $s1 = $s2 + $s3
  4: Orice: `sub $s1, $s2, $s3` $s1 = $s2 – $s3
  8: `j azi; pc = 16;` jump la functia azi(functia 16) ; pc -> program counter
  Maine:`lw $s1, 100($s2)` $s1 = Memory[$s2+100]
  16:azi: `sw $s1, 100($s2)` Memory[$s2+100] = $s1
  add $t0, $t1, $t2

Machine Language: R-type instr

- Instructions, like registers and words of data, are also 32 bits long
  - Example: `add $t0, $s1, $s2`
  - Registers have numbers: `$t0=8, $s1=17, $s2=18`

- Instruction Format:

type of instruction	s1	s2	t0	shift amount -> 0	opcode for operation
op	***rs***	***rt***	***rd***	***shamt***	***funct***
000000	10001	10010	01000	0000 10	0000
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Can you guess what the field names stand for?

Machine Language: I-type instr

- Consider the load-word and store-word instructions,
  - What would the regularity principle have us do?
  - New principle: *Good design demands a compromise*
- Introduce a new type of instruction format
  - I-type for data transfer instructions lw si sw + lb si sb
  - other format was R-type for register
- Example: `lw $t0, 32($s2)`

opcode for lw	value in S2	registru	offset
35	18	8	32

op	rs	rt	16 bit number

Control

- Decision making instructions
  - alter the control flow,
  - i.e., change the "next" instruction to be executed

- MIPS conditional branch instructions:

```
bne $t0, $t1, Label
beq $t0, $t1, Label
```

not equal

equal

- Example: if (i==j) h = i + j;

```
    bne $s0, $s1, Label   //if (i==j)
    add $s3, $s0, $s1 //h = i + j;
Label:....
```

folosim bne pt a nu sari la label

Control

- MIPS unconditional branch instructions:

```
j  label
```

- Example:

```
if (i!=j)                beq $s4, $s5, Lab1
    h=i+j;               add $s3, $s4, $s5
else                   j Lab2
    h=i-j;             Lab1:   sub $s3, $s4,
$s5
                       Lab2:   ...
```

- *Can you build a simple for loop?*

So far (including J-type instr):

- Instruction Meaning

```
add $s1,$s2,$s3   $s1 = $s2 + $s3
sub $s1,$s2,$s3   $s1 = $s2 - $s3
lw $s1,100($s2)   $s1 = Memory[$s2+100]
sw $s1,100($s2)   Memory[$s2+100] = $s1
bne $s4,$s5,L     Next instr. is at Label if $s4 ° $s5
beq $s4,$s5,L     Next instr. is at Label if $s4 = $s5
j Label        Next instr. is at Label
```

- Formats:

R	op	rs	rt	rd	shamt	funct
I	op	rs	rt	16 bit address		
J	op	26 bit address				

Control Flow

- We have: beq, bne, what about Branch-if-less-than?
- New instruction:

meaning:

```
if  $s1 < $s2 then
   $t0 = 1
      else
   $t0 = 0
```

```
slt $t0, $s1, $s2
```

set less than
si avem fix codul de mai din dreapta

- Can use this instruction to build "`blt $s1, $s2, Label`"
  — can now build general control structures

- Note that the assembler needs a register to do this,
  — use conventions for registers

Small Constants: immediates

- Small constants are used quite frequently (50% of operands)
  e.g., A = A + 5;
  B = B + 1;
  C = C - 18;

- MIPS Instructions: i stands for immediate -> operanzi nr intregis

```
   addi $29, $29, 4
slti $8,  $18, 10
andi $29, $29, 6
ori  $29, $29, 4
```

How about larger constants?

- We'd like to be able to load a 32 bit constant into a register
- Must use two instructions; new "load upper immediate" instruction

```
lui $t0, 1010101010101010
```

filled with zeros

1010101010101010	0000000000000000

- Then must get the lower order bits right, i.e.,

```
ori $t0, $t0, 1010101010101010
```

1010101010101010	0000000000000000
0000000000000000	1010101010101010

ori

1010101010101010	1010101010101010

Assembly Language vs. Machine Language

- Assembly provides convenient symbolic representation
  - much easier than writing down numbers
  - e.g., destination first
- Machine language is the underlying reality
  - e.g., destination is no longer first
- Assembly can provide '*pseudoinstructions*'
  - e.g., "`move $t0, $t1`" exists only in Assembly
  - would be implemented using "`add $t0,$t1,$zero`"
- When considering performance you should count real instructions

Addresses in Branches and Jumps

- Instructions:

  `bne $t4,$t5,Label` Next instruction is at Label if `$t4` ≠ `$t5`

  `beq $t4,$t5,Label` Next instruction is at Label if `$t4` = `$t5`

  `j Label` Next instruction is at Label

- Formats:

I	op	rs	rt	16 bit address
J	op	26 bit address		

- Addresses are not 32 bits
  — How do we handle this with load and store instructions?

Addresses in Branches

- Instructions:

  `bne $t4,$t5,Label` Next instruction is at Label if $t4 ≠ $t5

  `beq $t4,$t5,Label` Next instruction is at Label if $t4 = $t5

- Formats: use I-type

I	op	rs	rt	16 bit address

- Could specify a register (like lw and sw) and add it to address
  - use Instruction Address Register (PC = program counter)
  - most branches are local (principle of locality)
- Jump instructions just use high order bits of PC
  - address boundaries of 256 MB

To summarize:

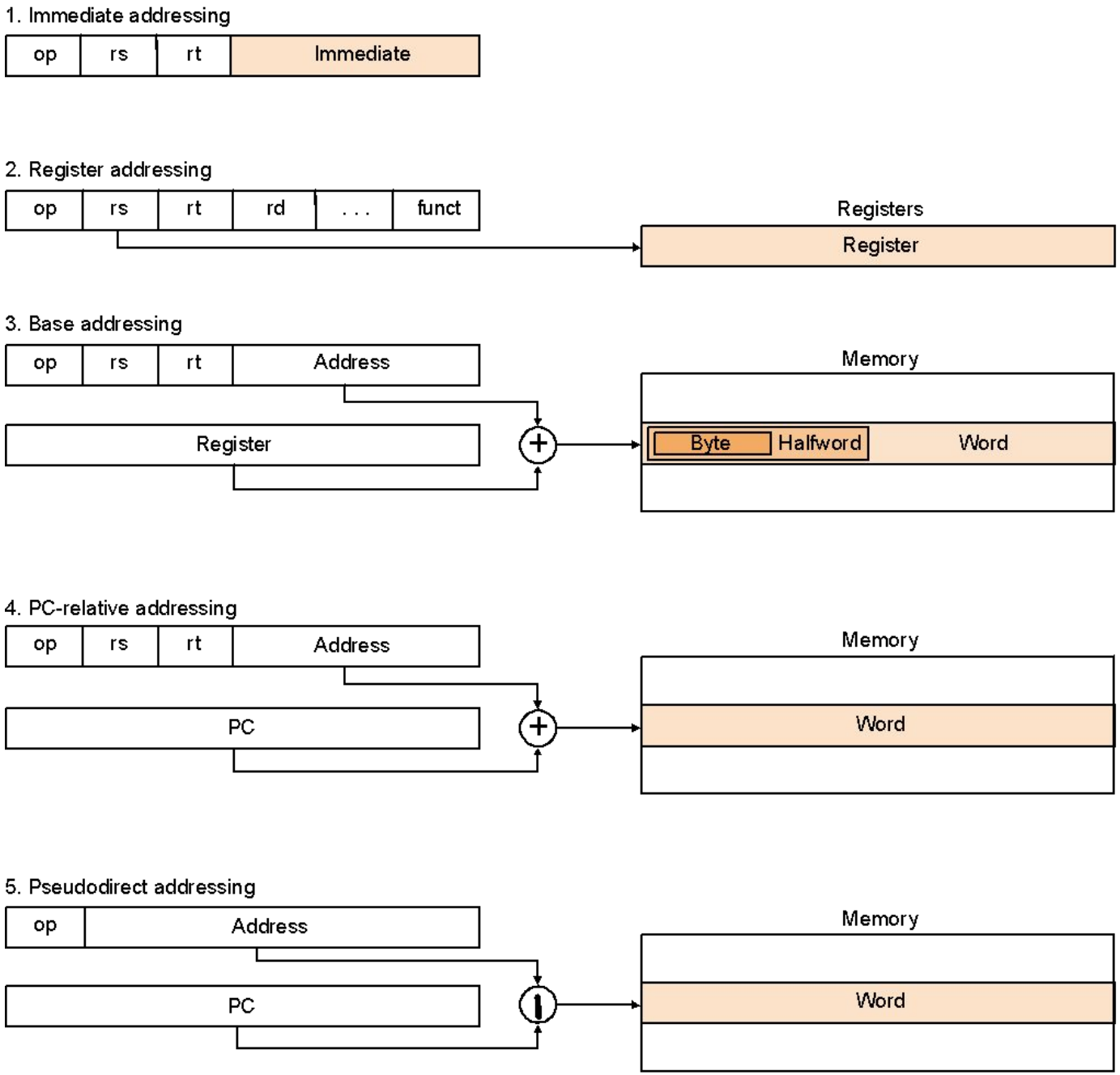
MIPS operands		
Name	**Example**	**Comments**
	`$s0-$s7, $t0-$t9, $zero,`	Fast locations for data. In MIPS, data must be in registers to perform
32 registers	`$a0-$a3, $v0-$v1, $gp,`	arithmetic. MIPS register $zero always equals 0. Register $at is
	`$fp, $sp, $ra, $at`	reserved for the assembler to handle large constants.
	Memory[0],	Accessed only by data transfer instructions. MIPS uses byte addresses, so
2^{30} memory	Memory[4], ...,	sequential words differ by 4. Memory holds data structures, such as arrays,
words	Memory[4294967292]	and spilled registers, such as those saved on procedure calls.

To summarize:

MIPS assembly language

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add $s1, $s2, $s3	$s1 = $s2 + $s3	Three operands; data in registers
	subtract	sub $s1, $s2, $s3	$s1 = $s2 - $s3	Three operands; data in registers
	add immediate	addi $s1, $s2, 100	$s1 = $s2 + 100	Used to add constants
Data transfer	load word	lw $s1, 100($s2)	$s1 = Memory[$s2 + 100]	Word from memory to register
	store word	sw $s1, 100($s2)	Memory[$s2 + 100] = $s1	Word from register to memory
	load byte	lb $s1, 100($s2)	$s1 = Memory[$s2 + 100]	Byte from memory to register
	store byte	sb $s1, 100($s2)	Memory[$s2 + 100] = $s1	Byte from register to memory
	load upper immediate	lui $s1, 100	$s1 = 100 * 2^{16}	Loads constant in upper 16 bits
Conditional branch	branch on equal	beq $s1, $s2, 25	if ($s1 == $s2) go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne $s1, $s2, 25	if ($s1 != $s2) go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt $s1, $s2, $s3	if ($s2 < $s3) $s1 = 1; else $s1 = 0	Compare less than; for beq, bne
	set less than immediate	slti $s1, $s2, 100	if ($s2 < 100) $s1 = 1; else $s1 = 0	Compare less than constant
Uncondi-tional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr $ra	go to $ra	For switch, procedure return
	jump and link	jal 2500	$ra = PC + 4; go to 10000	For procedure call

MIPS (3+2) addressing modes overview



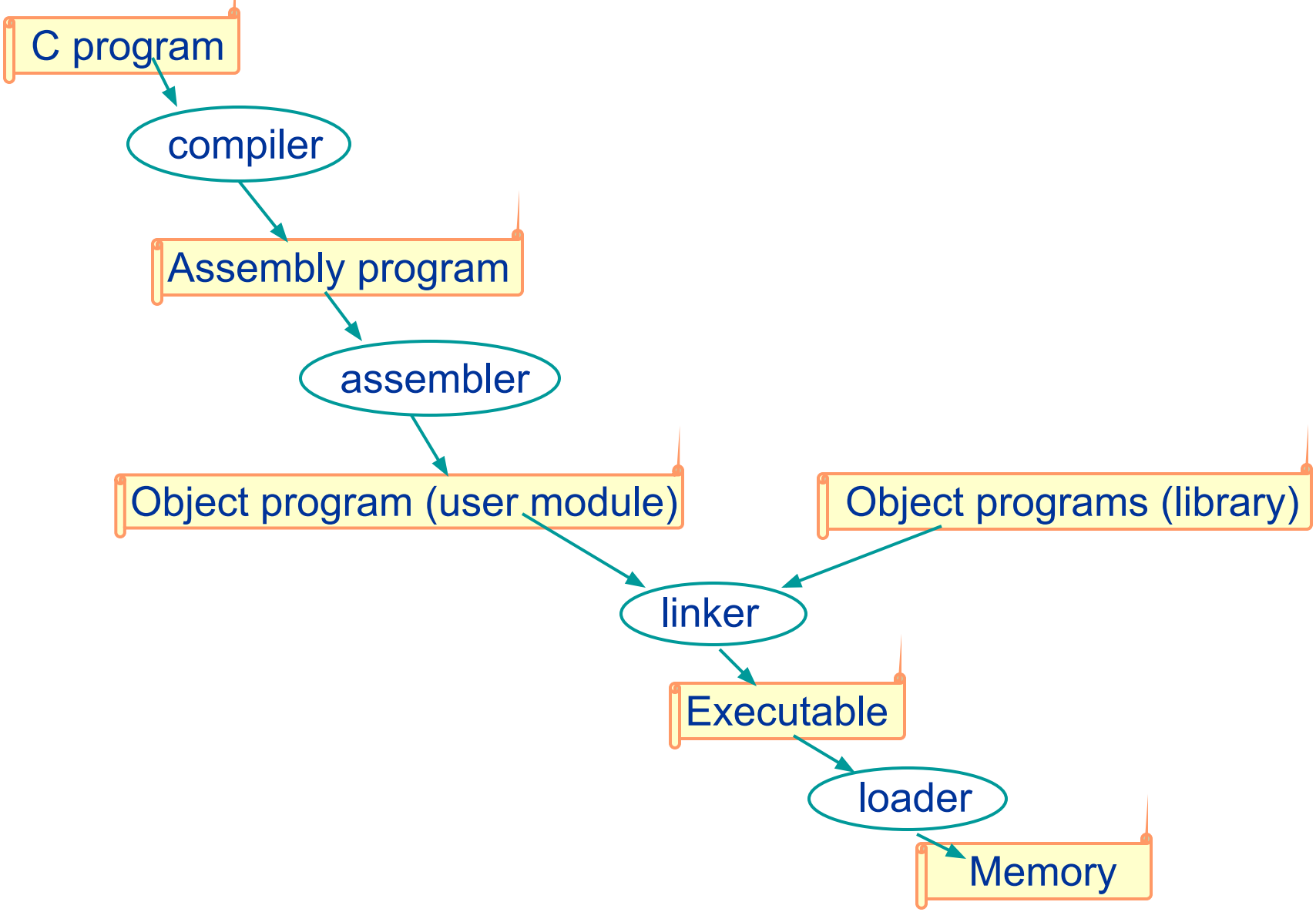
Starting a program

- **Compile** and **Assemble** C program

- **Link**
  - insert library code
  - determine addresses of data and instruction labels
  - relocation: patch addresses

- **Load** into memory
  - load text (code)
  - load data (global data)
  - initialize `$sp, $gp`
  - copy parameters to the main program onto the stack
  - jump to ‘start-up’ routine
    - copies parameters into `$ai` registers
    - call main

Starting a program



Compiler & Assembler

Compiler:

- Translates high-level source code into intermediate or low-level code.
- **Lexical Analysis**: Converts source code into tokens.
- **Syntax Analysis**: Constructs an Abstract Syntax Tree (AST) from tokens.
- **Semantic Analysis**: Ensures the semantic correctness of the AST.
- **Intermediate Code Generation**: Produces an intermediate representation.
- **Optimization**: Improves code performance and efficiency.
- **Code Generation**: Translates intermediate code into assembly or machine code.
- **Optional Further Optimization**: Enhances generated code for specific hardware.

Assembler:

- Translates assembly code into machine code.
- **Reading Assembly Code**: Interprets mnemonics and symbolic names.
- **Symbol Table Creation**: Maps symbolic names to addresses.
- **Address Resolution**: Determines memory addresses of instructions and data.
- **Instruction Translation**: Converts mnemonics into binary machine instructions.
- **Handling Directives**: Processes organizational commands.
- **Generating Object Files**: Produces object files with machine code and metadata.
- **Relocation**: Provides information for address adjustment during linking.

Linker & Loader

Linker:

- **Input**: Object files generated by the assembler.
- **Output**: A single executable or library file.
- The linker takes multiple object files as input and combines them into one cohesive program. These object files might be generated from different source files.

- Combines multiple object files into a single executable or library.
- Resolves symbol references and assigns final memory addresses.
- Handles both static and dynamic libraries.
- Generates a symbol table for the executable.

Loader:

- **Input**: The executable file produced by the linker.
- **Output**: The program loaded into memory, ready for execution.
- The loader reads the executable file and copies its contents into the memory.

- Loads the executable into memory for execution.
- Allocates memory for code, data, and stack segments.
- Performs final address relocations.
- Dynamically links shared libraries.
- Sets up the execution environment.
- Transfers control to the program's entry point.